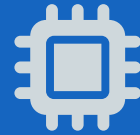


MODELAGEM COMPUTACIONAL



Máquinas de Turing - Parte 1

Prof. Dr. José Jairo de S. e Silva

Aula 07

Máquinas de Turing - Parte 1

OBJETIVOS DA AULA

🎯 Objetivos

Ao final desta aula, você deverá ser capaz de:

1. Reconhecer **limitações** dos modelos vistos até aqui (DFA, NFA, PDA) e identificar linguagens fora do seu alcance.
2. Descrever a **Máquina de Turing (MT)** intuitivamente: fita, cabeça de leitura, controle finito.
3. Aplicar a **definição formal** de uma MT como uma 7-tupla $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$.
4. Manipular **Descrições Instantâneas (IDs)** e a relação $\vdash_M$ entre passos de computação.
5. Distinguir linguagem **Recursivamente Enumerável (RE)** de linguagem **Recursiva (R) / Decidível**, identificando que **algoritmo = MT que sempre para**.
6. Construir MTs simples para linguagens como $\{0^n 1^n\}$, $\{w\#w\}$ e $\{a^n b^n c^n\}$.
7. Posicionar **R** e **RE** na hierarquia: Regulares $\subsetneq$ LLCs $\subsetneq$ R $\subsetneq$ RE.

1. POR QUE PRECISAMOS DE UM MODELO MAIS PODEROSO?

Recapitulando o que já temos

Até aqui, vimos três modelos de computação, em ordem **crescente** de poder:

Modelo	Memória	Reconhece
DFA / NFA	nenhuma (estado finito)	Linguagens regulares
PDA	uma pilha (LIFO)	Linguagens livres de contexto (LLCs)
?	?	?

Lacuna observada: problemas como **teste de primalidade** ($L_P = \{w \mid N(w) \text{ é primo}\}$) e linguagens como $\{a^n b^n c^n\}$ ou $\{ww \mid w \in \{0, 1\}^*\}$ **não** são livres de contexto. Nenhum PDA as reconhece. Mesmo assim, sabemos escrever programas em qualquer linguagem moderna que decidem essas tarefas.

DFAs e PDAs **não** são capazes de expressar todos os algoritmos. Precisamos de um modelo com **mais** que uma pilha. A solução é generalizar: dar ao autômato uma **fita** que pode ser lida, escrita e percorrida em **qualquer direção**.

⊘ Limitação do autômato com pilha: o que um PDA não consegue

Um PDA consegue reconhecer linguagens que exigem **contagem pareada de dois grupos** ou **estrutura recursiva** (parênteses balanceados, $\{a^n b^n\}$, palíndromos). Isso é possível porque a pilha permite empilhar enquanto lê o primeiro grupo e desempilhar enquanto lê o segundo. Toda a "memória" do PDA está concentrada nessa pilha.

Mas existem linguagens **muito naturais** que **nenhum PDA consegue reconhecer**:

Linguagem	Por que escapa do PDA
$\{a^n b^n c^n \mid n \geq 0\}$	Exige comparar três grupos de tamanho igual, mas uma pilha só consegue parear dois.
$\{w \# w \mid w \in \{0, 1\}^*\}$	Exige comparar duas cópias da mesma cadeia na ordem original; a pilha entrega na ordem reversa.
$\{ww \mid w \in \{0, 1\}^*\}$	Mesmo problema: copiar e comparar exige acesso bidirecional, que a pilha não tem.
$\{w \in \{a, b, c\}^* \mid \#_a(w) = \#_b(w) = \#_c(w)\}$	Exige três contadores simultâneos; a pilha só fornece um.

Esses problemas têm algo em comum: o PDA precisaria **voltar e ler** parte da entrada novamente, ou manter mais de uma "fila de memória" ao mesmo tempo. A pilha, sozinha, não dá conta.

▮ Esse resultado é formalmente provado pelo **Lema do Bombeamento para LLCs**, visto na Aula 6.

⊘ Por que a pilha não basta: a intuição

Considere a tentativa ingênua de reconhecer $L = \{a^n b^n c^n\}$ com um PDA:

1. Enquanto lê a s, empilha um marcador para cada a .
2. Enquanto lê b s, desempilha um marcador para cada b . **A pilha fica vazia** quando $n_a = n_b$.
3. Agora precisa verificar que o número de c s também é n , mas **não sobrou nada na pilha** para comparar.

Não importa como a equipe reordene a estratégia: sempre que a pilha for usada para verificar a igualdade de dois grupos, ela é **esvaziada nesse processo**, e o terceiro grupo fica sem referência.

Três propriedades que tornam o PDA limitado:

- **Memória de uso único:** o que é desempilhado some. Não há como ler a mesma "marcação" duas vezes.
- **Acesso só ao topo:** o PDA não vê o conteúdo do meio da pilha. Decisões dependem apenas do topo.
- **Entrada lida em uma única direção:** o cabeçote da entrada sempre avança. Não é possível "voltar e reler".

Para superar essas três limitações simultaneamente, é preciso um modelo onde **a própria memória possa ser revisitada quantas vezes for necessário**, em qualquer posição, lendo e escrevendo livremente. Esse modelo é a **Máquina de Turing**, que substitui a pilha por uma **fita bidirecional** sob controle do cabeçote.

Alan Turing e o modelo de 1936

Em 1936, [Alan Turing](#) propôs uma máquina abstrata com o objetivo de [formalizar a noção intuitiva de algoritmo](#). O artigo "*Sobre números computáveis, com uma aplicação ao Problema da Decisão*" introduziu o que hoje chamamos de [Máquina de Turing \(MT\)](#).

Componentes:

- Uma [fita](#) infinita dividida em células, inicialmente contendo a entrada e, depois dela, símbolos brancos ($\sqcup$).
- Um [cabeçote](#) que aponta para uma célula da fita e pode [ler](#) e [escrever](#) símbolos.
- Um [controle finito](#) (conjunto de estados) que decide, a cada passo, o que fazer.

A cada passo a MT:

1. Lê o símbolo sob o cabeçote.
2. Em função do estado atual e do símbolo lido, [escreve](#) um novo símbolo na célula.
3. [Move](#) o cabeçote uma posição para a esquerda (L) ou para a direita (R).
4. [Transita](#) para um novo estado.

Título original: "[On Computable Numbers, with an Application to the Entscheidungsproblem](#)", A. M. Turing, *Proc. London Math. Soc.* (1936).

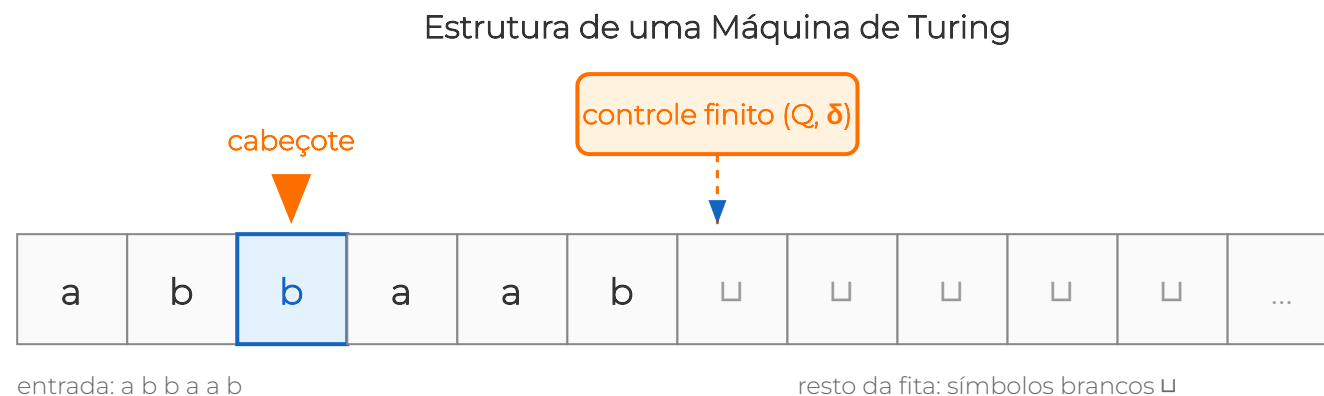
⦿ Para que serve a Máquina de Turing?

A MT **não foi inventada para ser executada em um computador**. Foi inventada para responder a uma pergunta teórica: *o que é, formalmente, um algoritmo?*. Quase 90 anos depois, ela continua sendo a referência usada nas seguintes situações:

- **Definir o conceito de algoritmo.** Toda vez que dizemos "isto pode ser computado", o que está por trás é uma MT (ou um modelo equivalente, pela Tese de Church-Turing).
- **Estabelecer limites do computável.** Existem problemas que **nenhum** programa, em **nenhuma** linguagem, consegue resolver. O exemplo clássico é o **Problema da Parada** (Aula 8): impossível decidir, no caso geral, se um programa qualquer vai terminar.
- **Fundamentar compiladores e verificadores.** Análise de tipos, verificação estática de código e *model checking* de hardware se apoiam em resultados de decidibilidade obtidos via MT.
- **Base da Complexidade Computacional.** As classes **P**, **NP**, **PSPACE** são definidas em termos de MTs com restrições de tempo e espaço. Sem MT, não existe formalismo para "este problema é difícil".
- **Modelar processos discretos.** Verificação de protocolos de comunicação, análise de sistemas embarcados, prova automática de propriedades de software seguro.

Em uma frase: a MT é a **régua** com a qual a Ciência da Computação mede o que é possível e o que não é. Toda linguagem de programação que você usa (Python, C, Rust) é equivalente a uma MT em poder, nem mais nem menos.

Visualização: fita, cabeçote e controle



A MT lê o símbolo sob o cabeçote, consulta a função δ no estado atual e decide: novo símbolo, movimento (L/R) e novo estado.

2. DEFINIÇÃO FORMAL

Definição 5.2.1: Máquina de Turing (Murilo, 2017)

Uma **Máquina de Turing** é uma **7-tupla**

$$M = (Q, \Sigma, \Gamma, \delta, q_0, B, F)$$

onde:

- Q : conjunto **finito** de estados;
- Σ : **alfabeto de entrada**;
- Γ : **alfabeto da fita**, com $\Sigma \subseteq \Gamma$;
- $\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times D$: função de transição **parcial**, com $D = \{L, R, S\}$;
- $q_0 \in Q$: estado **inicial**;
- $B \in \Gamma$: símbolo **branco** (marca células vazias da fita);
- $F \subseteq Q$: conjunto de estados **finais** (de aceitação).

A definição parece densa, mas três detalhes precisam de explicação separada (próximo slide): por que δ é chamada de **parcial**, por que ela **não** é definida em estados de F , e o que significa o movimento S .

i Três observações sobre a definição formal

(i) δ é uma função parcial.

Dizer que uma função é "parcial" significa que ela **não está obrigatoriamente definida em todos os pares de entrada**. No caso da MT, podem existir combinações (q, X) de estado e símbolo lido para as quais δ simplesmente **não tem regra**. Quando isso ocorre durante a execução, a MT **para**, sem aplicar nenhuma transição. Esse mecanismo é o que permite que a MT rejeite uma entrada: basta não definir transição para o par lido.

Exemplo. Em M_{01} , não existe $\delta(q_1, B)$. Se a MT chega em q_1 lendo branco, ela para sem aceitar: rejeição por parada não final.

(ii) δ não é definida para estados de F .

Quando a MT atinge um estado final $q \in F$, a computação **termina com aceitação**. Por isso, não faria sentido definir $\delta(q, X)$ para $q \in F$: a máquina já parou. É por isso que o domínio da função é escrito como $Q \setminus F$ ("estados de Q que não estão em F ") e não como Q inteiro.

Exemplo. Em M_{01} , $F = \{q_4\}$. Não há nenhuma linha da tabela com q_4 como estado de origem, porque q_4 é o ponto de parada.

(iii) O conjunto de movimentos $D = \{L, R, S\}$ inclui S (stay).

Além de mover à esquerda (L) ou à direita (R), o cabeçote pode **permanecer parado** na mesma célula (S , de *stay*). Isso é útil em situações em que a MT precisa **trocar de estado sem mover o cabeçote**, por exemplo logo após escrever um símbolo, antes de decidir o próximo passo.

Observação prática. Algumas formulações de MT (por exemplo, Sipser) permitem só L e R . O movimento S pode ser simulado por uma sequência R seguido de L , então as duas formulações são equivalentes em poder. Nesta disciplina usamos $\{L, R, S\}$ por ser mais conveniente para projetar MTs.

↩ Q - conjunto de estados em detalhe

Q é o "cérebro" da máquina: a memória **finita** que registra em que ponto do procedimento a MT está. **Nenhum estado** carrega a fita - tudo que a MT "lembra" cabe em um dos elementos de Q .

Características fundamentais:

- $|Q|$ é **finito**: por mais longa que seja a entrada, o controle não pode crescer.
- Cada $q \in Q$ representa um **modo de operação** (ex. "estou procurando o próximo **1**", "estou voltando para o início").
- Não é admissível "passar" informação entre estados via valores externos: tudo é codificado em q .

Em palavras simples: Q é como o **contador de programa** de um processador, só que com nomes ao invés de números. Cada estado responde à pergunta "o que a MT está fazendo agora?".

🔗 *Q* na prática: estados de M_{01}

M_{01} é a MT para $\{0^n1^n\}$, com $Q = \{q_0, q_1, q_2, q_3, q_4\}$.

A estratégia é "casar" cada 0 com um 1. Para isso, a MT usa dois **marcadores** auxiliares na fita: *X* substitui um 0 já processado e *Y* substitui um 1 já processado. A cada rodada, a MT marca um 0 como *X* e o respectivo 1 como *Y*.

Estado	O que esse estado faz na fita
q_0	Lê o primeiro símbolo não marcado da rodada. Se for 0, troca por <i>X</i> e avança para q_1 . Se for <i>Y</i> (não sobrou nenhum 0), avança para q_3 .
q_1	Caminha à direita , ignorando 0s e <i>Y</i> s, até encontrar um 1. Troca esse 1 por <i>Y</i> e passa para q_2 .
q_2	Caminha à esquerda , ignorando 0s e <i>Y</i> s, até reencontrar o <i>X</i> que foi posto na rodada. Volta uma posição à direita e retorna a q_0 para a próxima rodada.
q_3	Estado de verificação : atravessa os <i>Y</i> s restantes confirmando que não sobrou nenhum 1 não marcado. Se chegar ao branco <i>B</i> , transita para q_4 .
q_4	Estado final : a MT para aceitando.

A Σ vs Γ - dois alfabetos, um contém o outro

A MT tem **dois** alfabetos. A confusão é frequente; a distinção é essencial.

- Σ - **alfabeto de entrada**: símbolos que aparecem na **string original** w dada à máquina.
- Γ - **alfabeto da fita**: símbolos que **podem aparecer** em qualquer célula durante a execução.

A relação obrigatória é $\Sigma \subseteq \Gamma$, e tipicamente $\Gamma \supsetneq \Sigma$, com pelo menos um símbolo extra: o **branco** B . Também é comum incluir **marcadores** ($X, Y, \dot{0}, \dot{1}, \#$) que a MT usa para anotar onde já trabalhou.

No exemplo M_{01} :

$$\Sigma = \{0, 1\}, \quad \Gamma = \{0, 1, X, Y, B\}.$$

- $0, 1$ aparecem em Σ porque a entrada $w \in \{0, 1\}^*$.
- X e Y **não** podem aparecer na entrada - são marcadores internos que a MT usa para indicar "já processei este símbolo".
- B está em Γ para representar células vazias da fita; **nunca** aparece na entrada inicial.

Em palavras simples: Σ é o que o **usuário escreve**; Γ é o que a **MT pode rabiscar** durante o cálculo. Pense na entrada como uma folha em branco com algumas letras, e $\Gamma \setminus \Sigma$ como o **lápiz colorido extra** que só a MT usa.

↔ δ - a função de transição em detalhe

δ é o **programa** da MT. Ela toma o estado atual e o símbolo lido e devolve **três** coisas ao mesmo tempo:

$$\delta : (Q \setminus F) \times \Gamma \rightarrow Q \times \Gamma \times D, \quad D = \{L, R, S\}.$$

Notação de conjuntos: $Q \setminus F$ lê-se *Q menos F* e significa: todos os estados de Q que **não** estão em F . Por exemplo, se $Q = \{q_0, q_1, q_2, q_3, q_4\}$ e $F = \{q_4\}$, então $Q \setminus F = \{q_0, q_1, q_2, q_3\}$. Em palavras: δ só é definida para estados **não finais**.

A saída $\delta(q, X) = (q', X', d)$ codifica **uma única ação atômica**:

1. **Estado novo** q' : para onde o controle migra.
2. **Símbolo escrito** X' : o que sobrescreve a célula sob o cabeçote (pode ser igual a X).
3. **Movimento** d : L (esquerda), R (direita) ou S (parado).

Por que δ é parcial? Porque deixar $\delta(q, X)$ **indefinida** é o mecanismo de **rejeição** da MT. Se a máquina, no estado q , lê X e δ não tem entrada para esse par, a MT **para sem aceitar** - rejeição por parada não-final.

Por que δ não é definida em estados de F ? Porque atingir um estado final já encerra a computação com **aceitação**. Definir uma saída a partir de $q \in F$ seria contraditório: a máquina deve parar e aceitar imediatamente.

Em palavras simples: δ é uma **tabela** com colunas (estado atual, símbolo lido) e linhas (estado novo, símbolo escrito, direção). Linhas faltantes significam "rejeite aqui"; ler em uma linha de F não é necessário porque a máquina já parou.

▢ q_0 , B e F em detalhe

Estado inicial $q_0 \in Q$. É **único** e fixo na definição da MT. Toda computação começa nele, com o cabeçote sobre a **primeira célula da entrada**.

Símbolo branco $B \in \Gamma$. Marca células **vazias** da fita. Convenções:

- B aparece em **todas** as células à esquerda da entrada e à direita dela.
- $B \notin \Sigma$, de modo que a entrada nunca contém B originalmente.
- Quando a MT escreve B em uma célula, ela está "apagando" o que estava lá.

Estados finais (de aceitação) $F \subseteq Q$. Atingir qualquer $q \in F$ encerra a computação com **aceitação**.

- F pode ter **um** estado (como em M_{01} com $F = \{q_4\}$) ou vários.
- Em outras formulações da MT (estilo Sipser), aparece **um** q_{aceita} e **um** q_{rejeita} . Aqui usamos o estilo Murilo: rejeição ocorre quando δ é indefinida.

Em palavras simples: q_0 é o "main" do programa; B é o **vazio** que cerca a entrada; F é o **return success** da MT.

⚙️ Função de transição na prática

Quando $\delta(q, X) = (q', X', d)$, lê-se:

"Estando no estado q com a cabeça de leitura escaneando o símbolo X , transite para q' , sobrescreva X por X' e mova a cabeça segundo $d \in \{L, R, S\}$."

Notação no diagrama: uma seta de q_i para q_j rotulada $X/X' d$ representa $\delta(q_i, X) = (q_j, X', d)$. Convenções:

- $d = L$: cabeça move uma célula à **esquerda**.
- $d = R$: cabeça move uma célula à **direita**.
- $d = S$: cabeça **permanece** na mesma célula.

Quando a MT para: (1) ao atingir um estado de F (aceita); (2) ao chegar em um par (q, X) no qual δ **não está definida** (rejeita por parada não-final). Se nenhum dos dois ocorrer, a MT **executa indefinidamente** (loop infinito).

📷 Definição 5.2.2: Descrição Instantânea (ID)

Uma **Descrição Instantânea** (ID) de M é uma string $\alpha q \beta$ tal que $\alpha, \beta \in \Gamma^*$ e $q \in Q$. Ela descreve completamente a MT em um instante.

Interpretação de $X_1 X_2 \cdots X_{i-1} q X_i X_{i+1} \cdots X_n$:

- A fita contém $X_1 X_2 \cdots X_n$ (com B s à esquerda de X_1 e à direita de X_n);
- A máquina está no estado q ;
- A cabeça de leitura está sobre o símbolo X_i .

ID inicial sobre $w = w_1 w_2 \cdots w_n$: simplesmente $q_0 w_1 w_2 \cdots w_n$. A cabeça lê w_1 .

ID final: qualquer ID da forma $\alpha p \beta$ com $p \in F$ (estado de aceitação).

Em palavras simples: uma ID é um **instantâneo** da computação, capturando estado, fita e posição da cabeça em um único objeto matemático.

ID na prática: o que ela representa

Pense em uma ID como um **instantâneo da MT**. Ela registra três coisas ao mesmo tempo:

1. O que está escrito na fita.
2. Em que estado a máquina está agora.
3. Em qual célula o cabeçote está parado.

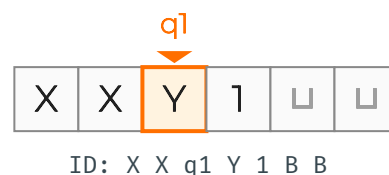
A convenção é escrever tudo isso em uma única linha de texto, inserindo o **nome do estado** logo à esquerda da célula que está sendo lida.

Exemplo. A ID XXq_1Y1BB significa:

- À esquerda do estado (XX): o que já passou pelo cabeçote.
- O estado q_1 aparece "colado" no Y : indica que o cabeçote está nessa célula.
- À direita ($Y1BB$): o símbolo lido agora (Y) seguido pelo restante da fita.

ID na prática: a versão visual

A mesma ID XXq_1Y1BB corresponde à seguinte fita:



A célula **em laranja** é a que o cabeçote está olhando: é o símbolo que δ vai usar como entrada no próximo passo.

Tudo à esquerda dela já foi visitado; tudo à direita ainda está por vir.

Em resumo: a notação $\alpha q \beta$ e a fita visual carregam exatamente a mesma informação. Em provas é mais prático usar a notação; em exemplos passo a passo, a fita visual ajuda a enxergar o que acontece.

→ Definição 5.2.3: passar de uma ID à próxima

A computação de uma MT é só uma **sequência de IDs**, uma depois da outra. Para passar de uma ID para a próxima, aplicamos a regra de δ uma única vez.

Quando isso acontece, dizemos que a primeira **leva à** segunda, e escrevemos:

$$I_1 \vdash_M I_2$$

Lê-se: a ID I_1 leva, em um passo, à ID I_2 .

Exemplo concreto. Suponha que a regra diga $\delta(q, Y) = (p, Y, L)$. Ou seja, no estado q lendo Y , a MT muda o estado para p , mantém o Y e move o cabeçote uma célula à esquerda. Então:

$$XXqY1B \vdash_M XpXY1B$$

O q "saltou" uma posição à esquerda do Y , indicando que agora o cabeçote olha o X daquela célula.

→ Vários passos seguidos e os três comportamentos possíveis

E muitos passos seguidos? Escrevemos:

$$I_1 \vdash_M^* I_2$$

para significar que I_2 é alcançada a partir de I_1 aplicando δ zero, uma ou várias vezes. Em palavras: a ID I_1 chega à ID I_2 em algum número de passos (incluindo zero, caso I_1 e I_2 sejam iguais).

O que pode acontecer quando M roda sobre uma entrada w ? Há exatamente três comportamentos possíveis:

1. **Aceita** w : depois de alguns passos, M chega em um estado final.
2. **Rejeita** w : M para porque não existe regra para o par (estado, símbolo) atual.
3. **Loop infinito**: M nunca para.

3. LINGUAGENS RECONHECIDAS E DECIDIDAS

A linguagem de uma MT

Toda MT, ao rodar sobre uma string w , ou aceita, ou rejeita, ou entra em loop. A pergunta natural é: **qual conjunto de strings essa MT aceita?**

Esse conjunto é o que chamamos de **linguagem da MT**.

Quando w é aceita. Começamos a MT na ID inicial (estado q_0 olhando o primeiro símbolo de w) e ela vai produzindo IDs. Se em algum momento o estado se torna **final**, dizemos que M aceita w .

Linguagem aceita por M , escrita $L(M)$. É simplesmente o conjunto de **todas** as strings sobre Σ que M aceita.

☰ Linguagens Recursivamente Enumeráveis

Definição 5.2.9 (Recursivamente Enumerável). Se existe alguma MT M tal que $L = L(M)$, dizemos que L é uma linguagem **Recursivamente Enumerável (RE)**.

Outro nome usual é "reconhecível por MT".

Cuidado com o "não aceita". Quando uma string w não pertence a $L(M)$, pode ser por dois motivos bem diferentes:

- M rodou, parou em algum momento, mas **sem** atingir um estado final (rejeitou); ou
- M **nunca parou** sobre w (ficou em loop).

Consequência prática: se uma MT apenas **reconhece** L (sem garantir que sempre para), e queremos saber se uma string w pertence ou não a L , podemos esperar para sempre sem nunca obter resposta. A MT aceita rapidamente todas as strings de L , mas para as strings de fora ela pode rodar indefinidamente, sem distinção entre "ainda vai rejeitar" e "nunca vai parar".

O que é um "algoritmo" de verdade

Uma MT pode "reconhecer" uma linguagem mesmo entrando em loop em algumas entradas. Mas, para usá-la na prática como um **algoritmo**, precisamos da garantia de que ela **sempre para**.

Afinal, um algoritmo de verdade tem que devolver uma resposta em tempo finito.

Definição 5.3.1 (Algoritmo). Uma MT que **sempre para**, qualquer que seja a entrada, é chamada de **algoritmo**. Ela pode parar aceitando ou parar rejeitando, mas nunca fica em loop.

Definição 5.3.2 (Decidir). Quando essa MT que sempre para reconhece exatamente as strings de L , dizemos que ela **decide** L .

Definição 5.3.3 (Linguagem Recursiva / Decidível). Uma linguagem que tem um algoritmo (no sentido acima) é chamada de **Recursiva** ou **Decidível**. O conjunto de todas elas é denotado por **R**.

RE versus Recursiva: comparação

Comparando os dois tipos de linguagem:

Comportamento da MT	L é apenas RE	L é Recursiva (R)
Se w está em L , a MT aceita?	sim	sim
Se w não está em L , a MT rejeita?	nem sempre	sempre
A MT pode entrar em loop?	pode	nunca

Toda linguagem Recursiva é também RE (basta ignorar a garantia de parada). A volta **não** vale: existem linguagens em RE que não são Recursivas.

O exemplo clássico de linguagem **RE mas não Recursiva** é o **Problema da Parada**. Uma MT consegue confirmar que outra para, mas não consegue, no caso geral, confirmar que ela *não* para. Veremos isso na Aula 8.

4. EXEMPLO INTRODUTÓRIO: $\{0^n 1^n \mid n \geq 1\}$

 MT para $L_{01} = \{0^n 1^n \mid n \geq 1\}$: a ideia

Esta é a **MT canônica de introdução**. A ideia é simples: a cada rodada, a MT marca um **0** à esquerda e um **1** à direita, formando um "par".

Se sobrar algum **0** ou **1** sem par, a entrada é rejeitada.

Como ela funciona, em palavras:

- Estado q_0 (início de rodada): troca o **0** corrente por X e vai procurar um **1**.
- Estado q_1 (procurando o **1**): anda para a direita; quando encontra o **1**, troca por Y e começa a voltar.
- Estado q_2 (voltando): anda para a esquerda até reencontrar o X que marcou o **0**.
- Quando, no início de uma rodada, só restam marcadores (Y), a MT passa ao estado q_3 (verificação) e, ao chegar no branco, aceita em q_4 .

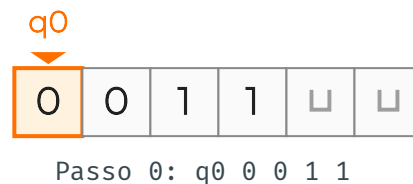
 MT para L_{01} : notação formal e tabela de transições

Notação resumida. O conjunto de estados é $Q = \{q_0, q_1, q_2, q_3, q_4\}$, o único estado final é q_4 , e o alfabeto da fita é $\{0, 1, X, Y, B\}$ (com B representando o branco).

De	Lê	Para	Escr.	M.
q_0	0	q_1	X	R
q_0	Y	q_3	Y	R
q_1	0	q_1	0	R
q_1	Y	q_1	Y	R
q_1	1	q_2	Y	L
q_2	0	q_2	0	L
q_2	Y	q_2	Y	L
q_2	X	q_0	X	R
q_3	Y	q_3	Y	R
q_3	B	q_4	B	R

► Computação de M_{01} sobre $w = 0011$ - ID inicial

Configuração inicial. A entrada **0011** é gravada nas células 1 a 4; demais células são $\sqcup$ (branco). O estado é q_0 e o cabeçote aponta para a célula 1.



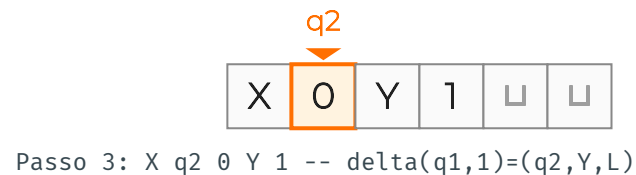
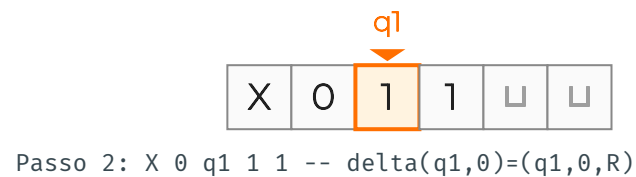
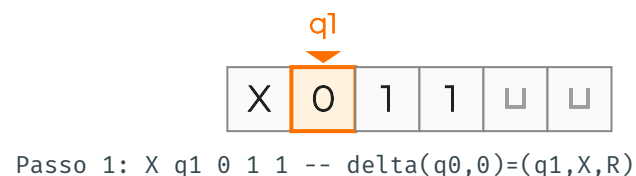
Próxima ação a ser executada. O par $(q_0, 0)$ aciona $\delta(q_0, 0) = (q_1, X, R)$:

1. **Escreve** X na célula 1 (marcando o primeiro 0).
2. **Move** o cabeçote uma célula para a direita.
3. **Migra** para o estado q_1 .

A cor **laranja** sempre destaca a célula sob o cabeçote. As próximas slides mostram a fita **depois de cada aplicação** de δ .

► M_{01} sobre 0011 - 1ª rodada (passos 1-4)

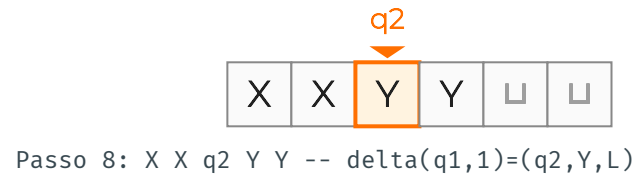
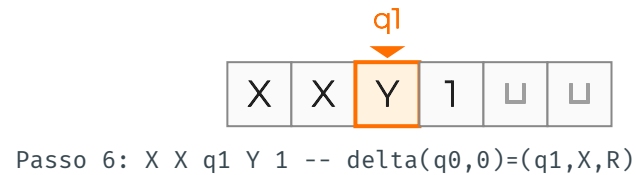
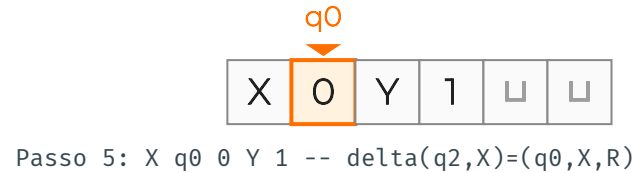
Marca o primeiro 0 como X , avança até o primeiro 1, marca-o como Y , volta.



Observação: ao final do passo 4, a primeira rodada está prestes a se fechar - o cabeçote acabou de chegar à célula 1, lendo X . A próxima aplicação de δ devolverá o controle a q_0 .

► M_{01} sobre 0011 - 2ª rodada (passos 5-8)

Volta a q_0 , marca o **segundo** 0, atravessa o Y já marcado, encontra o **segundo** 1 e marca-o como Y .



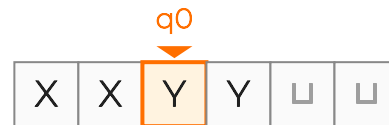
Observação: no passo 7, q_1 "pula" sobre o Y já marcado anteriormente - é por isso que existe a transição $\delta(q_1, Y) = (q_1, Y, R)$ na tabela.

► M_{01} sobre 0011 - volta + verificação (passos 9-12)

Termina a volta (passos 9-10), retorna ao q_0 que **agora encontra um Y** e dispara a fase de verificação (q_3).



Passo 9: X q_2 X Y Y -- $\delta(q_2, Y) = (q_2, Y, L)$



Passo 10: X X q_0 Y Y -- $\delta(q_2, X) = (q_0, X, R)$



Passo 11: X X Y q_3 Y -- $\delta(q_0, Y) = (q_3, Y, R)$

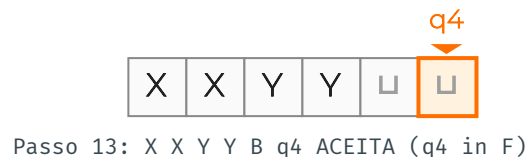


Passo 12: X X Y Y q_3 B -- $\delta(q_3, Y) = (q_3, Y, R)$

Por que muda de q_0 para q_3 no passo 11? Porque a tabela tem $\delta(q_0, Y) = (q_3, Y, R)$. Encontrar Y no estado q_0 é a evidência de que **todos os 0s já foram marcados**: a partir daqui, basta atravessar os Y s restantes até o branco.

🚩 M_{01} sobre 0011 - aceitação (passo 13)

A célula 5 contém $\sqcup$ (branco). Aplica-se $\delta(q_3, B) = (q_4, B, R)$, atinge $q_4 \in F$ e a MT **para aceitando**.



Resumo da computação completa $q_0 0011 \vdash_{M_{01}}^* XXYYB q_4$:

- Foram aplicados **13 passos** da relação $\vdash_{M_{01}}$;
- Foram realizadas **2 rodadas** marcando um **0** e depois um **1** (passos 0-4 e 5-8);
- Cada **rodada** combina os movimentos das transições da tabela em uma sequência que percorre a fita ida-e-volta uma vez;
- Como $q_4 \in F$, a string **0011** é **aceita**; portanto $0011 \in L(M_{01})$.

Em palavras simples: a fita começa com a entrada, vai sendo "riscada" em pares (**X** à esquerda, **Y** à direita) e, quando todos os símbolos foram marcados, a MT confirma que sobraram apenas marcadores e aceita.

⊗ M_{01} sobre $w = 001$ - rejeição (parte 1, passos 0-3)

Entrada inválida (001, com mais 0s que 1s). A 1ª rodada marca um 0 como X e o 1 como Y .



Passo 0: $q0$ 0 0 1 (inicial)



Passo 1: X $q1$ 0 1 -- $\text{delta}(q0,0)=(q1,X,R)$



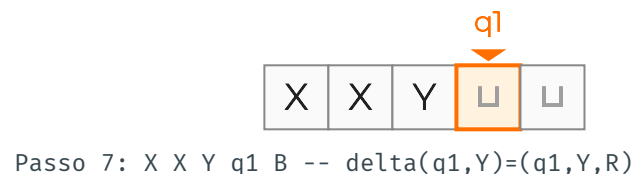
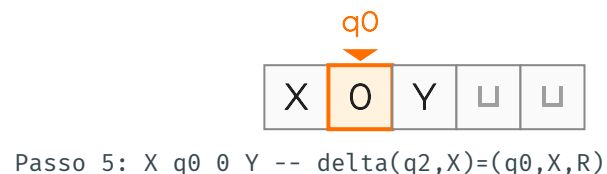
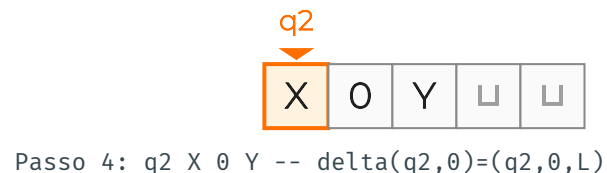
Passo 2: X 0 $q1$ 1 -- $\text{delta}(q1,0)=(q1,0,R)$



Passo 3: X $q2$ 0 Y -- $\text{delta}(q1,1)=(q2,Y,L)$

⊗ M_{01} sobre $w = 001$ - rejeição (parte 2, passos 4-7)

A 2ª rodada começa, marca o segundo 0 como X , mas não encontra outro 1 para parear: ao chegar no branco em q_1 , a MT trava.



Rejeição: no passo 7, q_1 lê $\sqcup$ (branco), mas **não existe** $\delta(q_1, B)$. A MT **para sem aceitar**: 001 é rejeitada.

 M_{01} no simulador online (turingmachinesimulator.com)

Cada transição vira **duas linhas**: a primeira diz "estado, símbolo lido" e a segunda "novo estado, símbolo escrito, movimento". O caractere `_` representa o branco; `>` move à direita, `<` à esquerda, `-` mantém o cabeçote parado.

name: 0n1n
init: q0
accept: q4

q0,0
q1,X,>

q0,Y
q3,Y,>

q1,0
q1,0,>

q1,Y
q1,Y,>

q1,1
q2,Y,<

q2,0
q2,0,<

q2,Y
q2,Y,<

q2,X
q0,X,>

q3,Y
q3,Y,>

q3,_
q4,_, -

Cole este código no simulador, clique **COMPILE**, insira **0011** como entrada e clique **play** para ver a animação. Para testar a rejeição, use **001** ou **0110**.

5. EXEMPLO: $\{w\#w \mid w \in \{0,1\}^*\}$

 Linguagem $L_{\#} = \{w\#w \mid w \in \{0,1\}^*\}$

Pertence a $L_{\#}$ qualquer string da forma $w\#w$, onde os dois lados do $\#$ são **idênticos**.

Exemplos:

- $\#$ ✓ (aqui $w = \varepsilon$)
- $0\#0$ ✓
- $11\#11$ ✓
- $0110\#0110$ ✓
- $01\#10$ ✗

Por que precisa de uma MT? Pode-se mostrar (via Lema do Bombeamento para LLCs) que $L_{\#}$ **não** é livre de contexto. Logo, nenhum PDA a reconhece. Mas uma MT **decide** $L_{\#}$, como veremos.

Estratégia: comparar os símbolos um a um, marcando os já visitados.

≡ Algoritmo de alto nível para $M_{\#}$

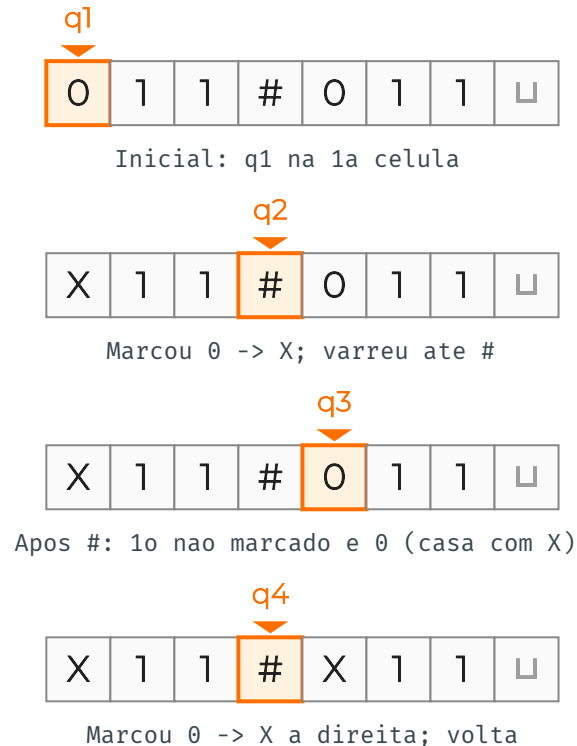
Sobre a entrada u :

1. Posicione o cabeçote no primeiro símbolo. Se for $\#$, vá ao passo 5.
2. **Marque** o símbolo atual (substitua **0** por **0̇** ou **1** por **1̇**) e mova à direita até encontrar $\#$.
3. Continue à direita até o **primeiro símbolo não marcado**. Se ele **coincide** com o que foi marcado, marque-o; caso contrário, **rejeite**.
4. Volte à esquerda até a primeira posição não marcada **antes** de $\#$. Repita o passo 2.
5. Verifique se sobrou algum símbolo não marcado depois do $\#$. Se sobrou, **rejeite**; caso contrário, **aceite**.

Em palavras simples: a MT alterna entre os dois lados do $\#$, riscando posições já comparadas. Termina quando todos os símbolos foram comparados sem inconsistência.

▶ Traço sobre 011#011 - 1ª comparação

Marcadores: **X** substitui **0** marcado; **Y** substitui **1** marcado. Estados de alto nível: q_1 = "marcar à esquerda", q_2 = "atravessar # até o 1º não marcado à direita", q_3 = "casar e marcar à direita", q_4 = "voltar até o 1º não marcado à esquerda".

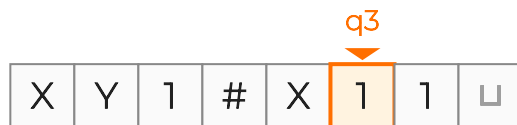


O que aconteceu nestes 4 passos: a MT consumiu uma posição de cada lado do #. O cabeçote já está voltando para procurar o próximo símbolo a ser comparado.

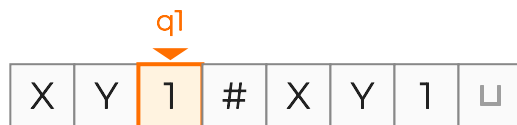
011#011 - 2ª e 3ª comparações



2a comparacao: 1o nao marcado a esquerda e 1



Marcou 1 -> Y; depois de #, 1o nao marcado tambem e 1



Marcou Y a direita; voltou ao 1o nao marcado: 1

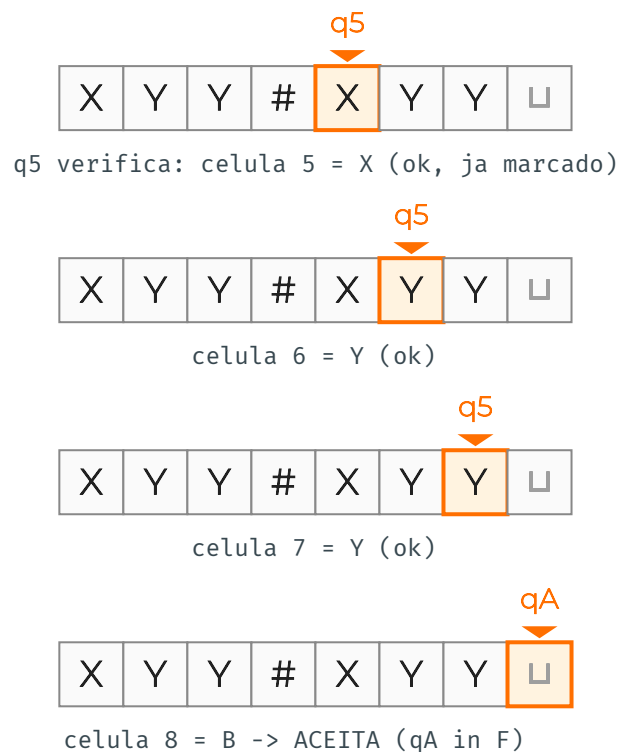


3a comparacao OK; q5 = nao ha mais nao marcado a esquerda

Cada **par** de snapshots corresponde a **uma comparação completa**: marca o símbolo à esquerda, casa com o primeiro não marcado à direita, volta para o próximo. Em três comparações esgotamos os dois lados.

011#011 - verificação final e aceitação

Após a 3ª comparação, q_5 varre o lado direito procurando algum não marcado.



Em palavras simples: se a verificação encontra o branco $\sqcup$ **sem** ter encontrado nenhum símbolo não marcado, é porque todos os símbolos do lado direito foram comparados com sucesso. Logo $011\#011 \in L_\#$.

⊗ Traço de rejeição sobre 01#10



Inicial: q_1 lê 0



Marcou 0 -> X; varreu ate #



Depois de #: 1o nao marcado e 1, mas esperava 0



Discordancia: q_R (parada nao final) -> REJEITA

A MT detecta a divergência **na primeira posição** que difere e termina em uma parada não-final (δ indefinida). Como **sempre para** (aceita ou rejeita), $M_{\#}$ **decide** $L_{\#}$.

 $M_{\#}$ no simulador (parte 1: cabeçalho, i e $m0$)

Esqueleto pronto para turingmachinesimulator.com. O slide seguinte traz a continuação. Os estados $m1$ e $n1$ ficam como exercício 8.

```
name: wcw
init: i
accept: A
```

```
i,0
m0,X,>
i,1
m1,Y,>
i,#
v,#,>
```

```
m0,0
m0,0,>
m0,1
m0,1,>
m0,X
m0,X,>
m0,Y
m0,Y,>
m0,#
n0,#,>
```

 $M_{\#}$ no simulador (parte 2: $n0$, b , c e v)

Continuação do esqueleto: busca do par à direita ($n0$), volta para a próxima comparação (b , c) e verificação final (v).

$n0, X$
 $n0, X, >$
 $n0, Y$
 $n0, Y, >$
 $n0, 0$
 $b, X, <$

 b, X
 $b, X, <$
 b, Y
 $b, Y, <$
 $b, \#$
 $c, \#, <$

c, X
 $c, X, <$
 c, Y
 $c, Y, <$
 $c, 0$
 $i, 0, -$
 $c, 1$
 $i, 1, -$

 v, X
 $v, X, >$
 v, Y
 $v, Y, >$
 $v, _$
 $A, _, -$

Cole as duas partes em sequência, complete $m1$ e $n1$ (analogia com $m0$ e $n0$, trocando $0 \leftrightarrow 1$ e $X \leftrightarrow Y$) e teste com $011\#011$ (aceita) e $01\#10$ (rejeita).

**6. EXEMPLO: $\{a^n b^n c^n \mid n \geq 1\}$ - exigindo poder além das
LLCs**

$$\pencil L_{abc} = \{a^n b^n c^n \mid n \geq 0\}$$

Strings com **igual número** de *as*, *bs* e *cs*, na ordem $a^*b^*c^*$.

Exemplos: ε ✓, *abc* ✓, *aabbcc* ✓, *aaabbbccc* ✓, *abbc* ✗, *aabc* ✗.

Por que MT, e não PDA? Uma pilha basta para **comparar dois grupos** ($a^n b^n$): empilha *as*, desempilha um *b* para cada. Mas para **três grupos**, depois de comparar *as* com *bs*, a pilha está vazia: não sobra nada para comparar com os *cs*.

Estratégia da MT: percorre a fita **várias vezes**. Em cada passada, marca **um a**, **um b** e **um c**. Quando todos foram marcados, aceita.

≡ Algoritmo de alto nível para M_{abc}

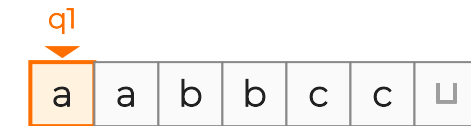
Sobre a entrada u :

1. Faça uma passada preliminar para verificar se a entrada está em $a^*b^*c^*$. Caso contrário, **rejeite**.
2. Volte ao início.
3. Marque o **primeiro** a não marcado. Se não há a não marcado, vá ao passo 6.
4. Avance até o **primeiro** b não marcado e marque-o. Se não há, **rejeite**.
5. Avance até o **primeiro** c não marcado e marque-o. Se não há, **rejeite**. Volte ao passo 2.
6. Verifique se sobrou algum b ou c não marcado. Se sim, **rejeite**; senão, **aceite**.

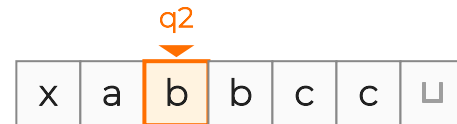
Em palavras simples: cada iteração elimina exatamente um a , um b e um c . Aceita quando os três grupos esgotam **simultaneamente**.

▶ Traço sobre *aabbcc* - iteração 1

Marcadores: $x = a$ marcado, $y = b$ marcado, $z = c$ marcado. Estados de alto nível: q_1 = procurar *a*, q_2 = procurar *b*, q_3 = procurar *c*, q_4 = voltar ao início, q_5 = verificar.



Inicial: q_1 procura 1o a



Marcou a -> x; avançou ate o 1o b



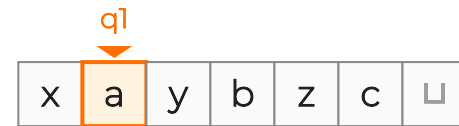
Marcou b -> y; avançou ate o 1o c



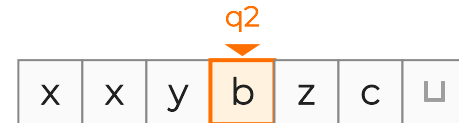
Marcou c -> z; q_4 voltou ao inicio

Em uma iteração a MT consome exatamente um *a*, um *b* e um *c*. Repare como o cabeçote percorre a fita inteira: ida (marcando) e volta (procurando o próximo *a*).

▶ *aabbcc* - iteração 2



q1 acha o 2o a (1o nao marcado)



Marcou a -> x; foi ate o 1o b nao marcado



Marcou b -> y; foi ate o 1o c nao marcado



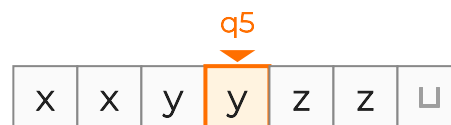
Marcou c -> z; voltou para nova rodada

Após a iteração 2, **todos os três grupos** ficaram completamente marcados. Quando q_1 procurar pelo próximo **a** não marcado, não encontrará nenhum: o controle migra para a fase de verificação q_5 .

🚩 *aabbcc* - verificação e aceitação



q1 não acha a -> entra em q5 (verificação)



q5 atravessa: sem b não marcado



q5 atravessa: sem c não marcado

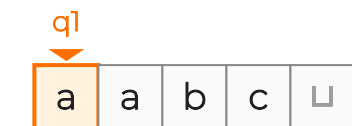


celula 7 = B -> ACEITA (qA in F)

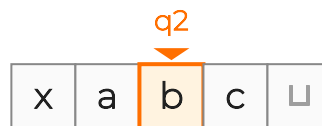
Em palavras simples: uma iteração da MT é **uma travessia da fita** que rabisca exatamente um símbolo de cada grupo. Aceita quando, no início de uma nova rodada, não sobra mais nenhum **a** não marcado **e** a verificação confirma que **bs** e **cs** também acabaram.

⊗ Rejeição sobre *aabc* (parte 1: 1ª iteração)

A entrada *aabc* tem *dois* *a*s mas apenas *um* *b* e *um* *c*. A 1ª iteração marca um de cada e termina normalmente.



Inicial: q1 procura 1o a



Marcou a -> x; foi ate b



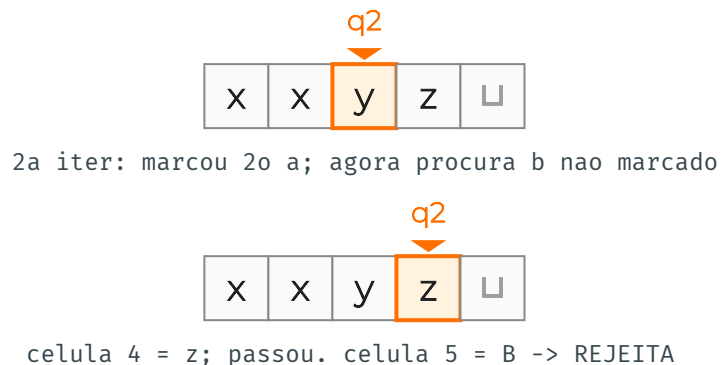
Marcou b -> y; foi ate c



Marcou c -> z; voltou (iter 1 ok)

⊗ Rejeição sobre abc (parte 2: 2ª iteração e falha)

A 2ª iteração marca o segundo a , mas não há um segundo b disponível. Em q_2 , o cabeçote ultrapassa o z marcado e chega no branco, onde δ não está definida.



Por que rejeita? Em q_2 , ao chegar em $\sqcup$ sem ter encontrado b não marcado, δ não está definida em (q_2, B) : parada não-final. A MT decide L_{abc} , aceitando quando os ns coincidem e rejeitando em qualquer outro caso.

 M_{abc} no simulador (parte 1: marcação)

Esqueleto pronto para turingmachinesimulator.com. Marcadores: x para a marcado, y para b marcado, z para c marcado. O slide seguinte traz a volta e a verificação final.

name: anbncn
init: q1
accept: qA

q1,a
q2,x,>
q1,x
q1,x,>
q1,y
q5,y,>

q2,a
q2,a,>
q2,y
q2,y,>
q2,b
q3,y,>

q3,b
q3,b,>
q3,z
q3,z,>
q3,c
q4,z,<

M_{abc} no simulador (parte 2: volta e verificação)

Continuação: q_4 caminha à esquerda até reencontrar o branco e devolve o controle a q_1 ; q_5 percorre os marcadores remanescentes e aceita ao chegar no branco da direita.

q_4, a
 $q_4, a, <$
 q_4, b
 $q_4, b, <$
 q_4, c
 $q_4, c, <$
 q_4, x
 $q_4, x, <$
 q_4, y
 $q_4, y, <$
 q_4, z
 $q_4, z, <$
 $q_4, _$
 $q_1, _, >$

q_5, y
 $q_5, y, >$
 q_5, z
 $q_5, z, >$
 $q_5, _$
 $q_A, _, -$

Cole as duas partes em sequência no simulador e teste com abc e $aabbcc$ (aceita) e com $aabc$, $abbcc$ ou $acbc$ (rejeita).

7. VARIANTES DE MT (PRÉVIA)

MTs com múltiplas fitas

Uma MT com k fitas tem k **cabeçotes independentes**, um por fita. A função de transição se generaliza para

$$\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k,$$

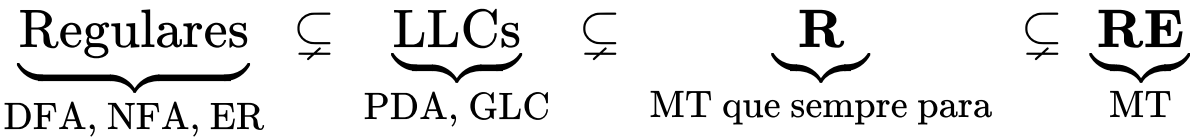
onde S ("stay") permite manter o cabeçote parado. A entrada começa na fita 1; as demais começam em branco.

Teorema (a ser visto na Aula 8): para toda MT multi-fita M , existe uma MT de **fita única** M' com $L(M') = L(M)$.

Múltiplas fitas tornam a programação de uma MT muito mais clara, mas **não aumentam o poder computacional** - apenas a eficiência.

8. HIERARQUIA ATUALIZADA

Onde a MT se encaixa?



Classe	Modelo	Exemplo dentro	Exemplo fora
Regulares	DFA, NFA, ER	$(01)^*$	$\{0^n 1^n\}$
LLCs	PDA, GLC	$\{0^n 1^n\}$	$\{a^n b^n c^n\}$
R (Recursivas)	MT que sempre para	$\{a^n b^n c^n\}, \{w\#w\}$	A_{TM} (Aula 8)
RE (Rec. Enumeráveis)	MT	A_{TM}	linguagens fora de RE (Aula 8)

Mensagem central: ao adicionarmos uma fita ilimitada e bidirecional, saímos da hierarquia "estado finito + memória restrita" e entramos no domínio da **computação geral** - capaz de expressar qualquer algoritmo (Tese de Church-Turing, Aula 8).

9. EXERCÍCIOS

☑ Questões objetivas

1. Dado $Q = \{q_0, q_1, q_2, q_3, q_4\}$ e $F = \{q_4\}$, escreva $Q \setminus F$.

▮ *Dica.* Aplique a definição de diferença de conjuntos: pegue todos os elementos de Q que **não** estão em F .

2. A entrada de M_{01} é gravada em $\Sigma = \{0, 1\}$ ou em $\Gamma = \{0, 1, X, Y, B\}$? Explique em uma frase.

▮ *Dica.* Lembre: Σ é o que o usuário escreve; Γ é o que a MT pode usar enquanto roda.

3. A partir da ID $X q_1 0 1 B$ e da transição $\delta(q_1, 0) = (q_1, 0, R)$, escreva a próxima ID.

▮ *Dica.* O cabeçote escreve, depois move. Como o movimento é R , o estado "pula" uma posição à direita.

4. Verdadeiro ou falso? *Toda linguagem regular pertence a $\mathbf{R}$.* Justifique.

▮ *Dica.* Todo DFA pode ser transformado em uma MT que sempre para. Em um passo por símbolo, ela ou aceita ou rejeita.

5. Verdadeiro ou falso? *Toda linguagem em $\mathbf{R}$ é livre de contexto.* Justifique.

▮ *Dica.* Pense em $\{a^n b^n c^n\}$. Você consegue uma MT para ela? E um PDA?

? Questões discursivas

6. Escreva o ID inicial de M_{01} sobre $w = 000111$ e calcule a sequência $\vdash_{M_{01}}^*$ até o ID final.

Dica. Mesma estratégia do traço da aula com $w = 0011$, mas com 3 rodadas de marcação em vez de 2. Deve dar 19 passos no total.

7. (Murilo, Ex. 5.5) Apresente uma MT M_3 que aceite, mas não decida, $L_3 = \{w \mid N(w) \text{ é múltiplo de } 3\}$.

Dica. Aceitar mas não decidir significa: quando $w \in L_3$, a MT precisa parar e aceitar; quando $w \notin L_3$, pode entrar em loop. Pense em uma MT que "tenta" todas as divisões.

8. Complete o esqueleto da MT do $w\#w$ (slide do simulador): escreva os estados $m1$ e $n1$ que tratam o caso em que a esquerda começa com 1, e teste no simulador.

Dica. As regras são analogias diretas das de $m0$ e $n0$. Troque $0 \rightarrow 1$ e $X \rightarrow Y$.

9. (Murilo, Ex. 5.8) Forneça uma MT M_{COPY} que, dada x na fita, deixe xx e aceite.

Dica. Marque cada símbolo de x por vez, ande até o fim da fita, escreva uma cópia e volte para marcar o próximo. É a primeira MT que computa uma função, não só decide.

10. (Murilo, Ex. 5.7) Mostre que $\mathbf{R} \subseteq \mathbf{RE}$.

Dica. Toda MT que sempre para também reconhece. Basta argumentar que a definição de "reconhecer" é satisfeita por toda MT que decide.

🚩 Encerramento da Aula 7

O que vimos:

- Por que precisamos ir além de PDAs (problemas como teste de primalidade, $a^n b^n c^n$).
- Definição formal da MT como 7-tupla $(Q, \Sigma, \Gamma, \delta, q_0, B, F)$.
- **Descrições Instantâneas (IDs)** e relação $\vdash_M, \vdash_M^*$.
- **RE** (Recursivamente Enumeráveis) $\times$ **R** (Recursivas/Decidíveis).
- **Definição:** Algoritmo = MT que sempre para.
- Construção de MTs para $\{0^n 1^n\}$, $\{w \# w\}$ e $\{a^n b^n c^n\}$.
- Variantes (multi-fita, não determinismo) e equivalências (preview).
- Hierarquia: Regulares $\subsetneq$ LLCs $\subsetneq$ **R** $\subsetneq$ **RE**.

Próxima aula (Parte 2):

- **Tese de Church-Turing:** equivalência MT $\leftrightarrow$ linguagens de programação.
- Codificação de MT como string $\langle M \rangle$.
- **Máquina de Turing Universal.**
- **Problema da Parada** (A_{TM}) e indecidibilidade.

Obrigado!

Dúvidas?